

开发手册

SOM V3.4 二次开发手册

文档版本号 (V1.8)



北京启创远景科技有限公司

文档属性

文档主题:	开发手册-SOM v3.4 二次开发说明书		
文档编号:			
文档创建人:	彭莉		
创建日期:	2017 年 10 月 1 日		
保密等级:	☐ 绝密级	☐ 机密级	☒ 秘密级
文档目的:	☒ 提交给客户		☐ 内部参阅和存档

文档变更

版 本	修订日期	修 订 人	描 述
V1.0	2017 年 10 月 1 日	彭莉	创建
V1.1	2017 年 10 月 13 日	彭莉	修改格式, 更新描述不妥之处
V1.2	2017 年 10 月 27 号	施晔	增加 6.2.2 节、6.4.1 节内容, 修改调整不当内容
V1.3	2017 年 11 月 10 日	施晔	增加对 CgetOppNums 函数的说明, 修改球车方向角示图 6.1 的错误
V1.4	2017 年 11 月 17 日	刘然	worldmodel lib 库内容补充
V1.5	2017 年 12 月 25 日	施晔	添加附录 5
V1.6	2018 年 3 月 5 日	翁润聪	修改附录 5, 添加附录 6

V1.7	2018 年 3 月 26 日	王晓彬	版本修订
V1.8	2018 年 9 月 7 日	张晏阳	版本修订

目录

目录	3
1 基本准备	7
1.1 文档说明	7
1.2 运行环境	7
1.2.1 软件系统安装	7
1.2.2 LUA 开发环境搭建	9
1.2.3 C++开发环境搭建	10
1.3 适配硬件	16
1.4 名词解释	16
2 系统框架	18
2.1 ES 系列机器人整体系统框架	18
2.2 SOM 软件系统架构	18
2.2.1 前端	19
2.2.2 后端	19
3 策略脚本层（LUA）框架	20
3.1 战术配合层（play）	20
3.1.1 定义	20

3.1.2 说明	21
3.2 任务层 (task)	23
3.2.1 定义	23
3.2.2 说明	23
3.4 技能层 (skill)	24
3.4.1 定义	24
3.4.2 说明	25
3.4 基础函数	25
4 决策规划层 (C++) 扩展	26
4.1 扩展内容	26
4.2 静态库	26
4.3 工具包	26
4.4 自定义技能 c++ 框架	26
4.5 用户技能 dll 放置目录	27
5 实战案例	28
5.1 官方 task 实战场景	28
5.1.1 战术设计	28
5.1.2 战术分解	29
5.1.3 完整示例 play 代码	30
5.2 自定义 task 实战场景	32
5.2.1 战术设计	32
5.2.2 战术分解	33
5.2.3 完整 play 代码	33
6 附件	35
6.1 附件 1: task 函数说明	35
6.1.1 KickerTask	35
6.1.2 ReceiverTask	35
6.1.3 TierTask	36
6.1.4 DefenderTask	36
6.1.5 MiddleTask	37

6.1.6	GoalieTask	37
6.1.7	GetBall	38
6.1.8	Goalie	38
6.1.9	GoRecePos	38
6.1.10	RobotHalt	39
6.1.11	NormalDef	39
6.1.12	PassBall	39
6.1.13	ReceiveBall	39
6.1.14	Stop	40
6.1.15	Shoot	40
6.1.16	RefDef	40
6.1.17	GotoPos	41
6.1.18	PenaltyDef	41
6.1.19	PenaltyKick	41
6.2	附件 2: 基础函数说明	41
6.2.1	比赛信息: 我方球员	42
6.2.2	比赛信息: 敌方球员	45
6.2.3	比赛信息: 球	48
6.2.4	比赛信息: 比赛条件	50
6.3	附件 3: worldmodel lib 库说明	50
6.3.1	get_ball_pos	51
6.3.2	get_our_player_pos	51
6.3.4	get_opp_player_pos	51
6.3.4	get_ball_vel	51
6.3.5	get_our_player_v	52
6.3.6	get_opp_player_dir	52
6.3.7	get_our_player_dir	52
6.3.8	get_our_goalie	52
6.3.9	get_opp_goalie	52
6.3.10	get_our_exist_id	53

6.3.11	get_opp_exist_id	53
6.4	附件 4: utils 工具包说明	53
6.4.1	orientate	53
6.4.2	target_pos	54
6.4.3	needKick	54
6.4.4	isPass	54
6.4.5	needCb	55
6.4.6	isChipPass	55
6.4.7	kickPower	55
6.5	附件 5: 开始写自己的代码前你需要知道的信息	56
6.5.1	足球机器人场地的坐标系建立	56
6.5.2	足球机器人比赛场景的定义	56
6.6	附件 6: 官方 task 函数源码 (含源码包)	58
6.6.1	GetBall 拿球函数	58
6.6.2	goalie 守门员函数	58
6.6.3	GoReceivePos 到接球点函数	59
6.6.4	GotoPos 跑位函数	59
6.6.5	NormalDef 普通防守函数	59
6.6.6	PassBall 传球函数	59
6.6.7	PenaltyDef 点球防守函数	59
6.6.8	PenaltyKick 发点球函数	59
6.6.9	ReceiveBall 接传球函数	59
6.6.10	RefDef 战术防守函数	59
6.6.11	Shoot 射门函数	59

1 基本准备

1.1 文档说明

本文档为 SOM-V3.4 的二次开发手册，将说明如何使用 SOM-V3.4 进行策略脚本（LUA）、决策算法（C++）的二次开发。

1.2 运行环境

在使用 SOM-V3.4 进行二次开发前，请确认已准备以下软硬件环境

操作系统	Windows 7/8/10
软件系统	SOM V3.4.1 下载地址: https://pan.baidu.com/s/1qXCSEfi
LUA 编辑工具	LUA 版本为 5.1，LUA 编辑工具为 SublimeText3
C++开发 IDE	VS2013 Ultimate
二次开发 lib 库	worldmodel_lib
二次开发工具包	utils
二次开发 DEMO 程序	demo.zip（包含: worldmodel_lib, utils, def）
环境包	环境包内包括（二次开发 lib 库，二次开发工具包，二次开发 DEMO 程序），随软件包一同下载即可。

1.2.1 软件系统安装

SOM V3.4 安装步骤如下：

1. 从网站下载 SOM V3.4 免安装压缩文件。
2. 在本地解压，如图 1-1

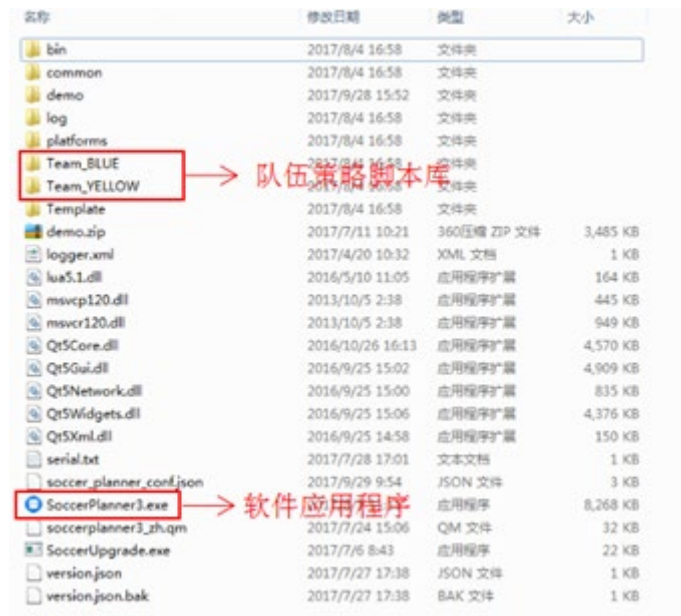


图 1-1 SOM V3.4 解压后文件目录

3. 解压后，对根目录下的 SoccerPlanner3.exe 应用程序右击属性，选择兼容性，勾选“以管理员身份运行此程序”，如图 1-2

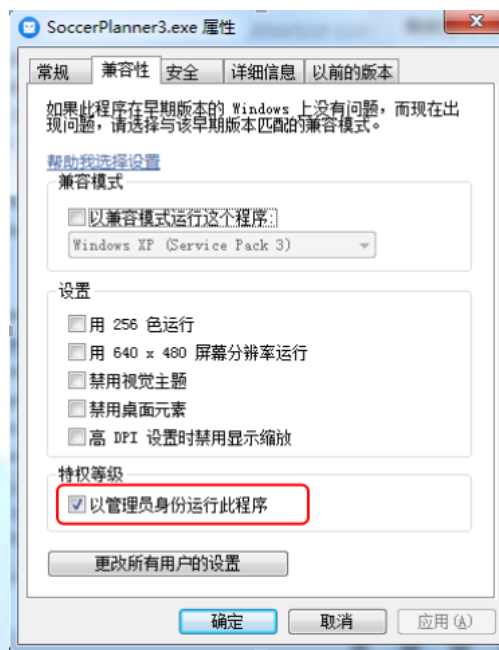


图 1-2 兼容性设置

4. 打开 Team_BLUE 或 Team_YELLOW 文件夹，如图 1-3

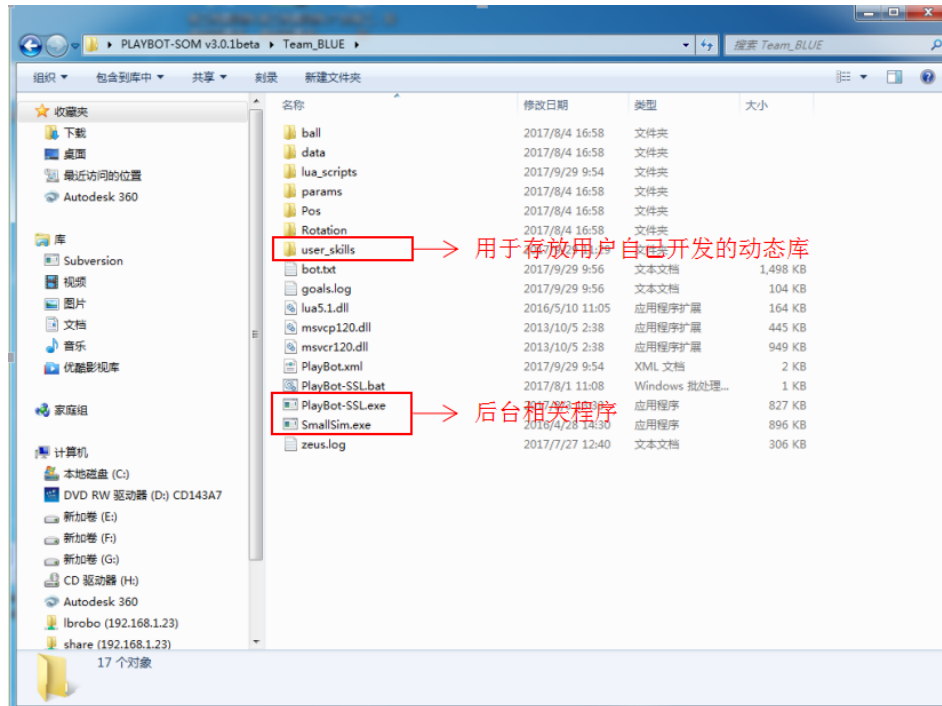


图 1-3 单个队伍文件目录

5. 分别对 Team_BLUE 和 Team_YELLOW 文件夹内的 PlayBot-SSL.exe 和 SmallSim.exe 两个应用程序进行与 SoccerPlanner3.exe 相同的设置。
6. 详细内容可参考《SOM-V3.4 策略系统操作手册》，手册在同时提供的软件系统包中。

1.2.2 LUA 开发环境搭建

- SOM V3.4 软件已定义了 LUA 战术脚本框架，软件会自行解析框架中内容，用户无需单独搭建第三方环境，必须严格按照 LUA 战术脚本框架编写。（具体 LUA 战术框架请参见 [5.1.1](#)）
- 强烈建议用户使用 SublimeText3 编辑器编写 LUA 脚本，此编辑器可以在网络上免费下载并安装。
- 用户编写 LUA 脚本必须严格按照 SOM V3.4 软件内含 LUA 环境的框架格式及语法要求。
- 用户如果希望能进一步掌握 LUA 的语法系统，可以参考查阅《LUA 中文教程》。建议初次使用 LUA 的用户详细阅读“类型和值”、“表达式”、“基本语法”、“函数”章节。使用独立的 LUA 开发工具 LUAEdit-V6.3.0。该工具在网上有丰富的资料，用户可以自行学习。（此内容已超出 SOM-V3.4 软件开发手册范围，此处不做详细说明）

1.2.3 C++开发环境搭建

C++开发环境搭建步骤如下：

1. 在 <https://pan.baidu.com/s/1cvu1R4> 网盘中下载 MicroSoft Visual Studio 2013（以下简称 VS2013）。

说明：VS2013 的版本众多，强烈建议用户由网盘中下载。

2. 安装 VS2013，直接按提示选择即可，此版本为官方免费版，无需密钥激活。
3. 安装完成后，新建项目

- 1) 单击 File->New->Project，如图 1-4

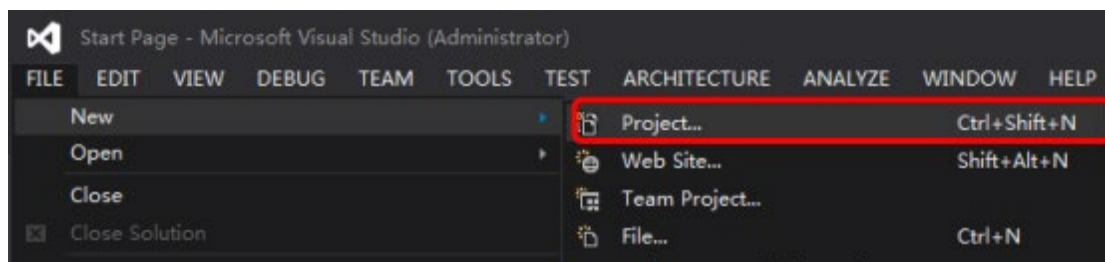


图 1-4 创建新 project

- 2) 弹出如下界面（图 1-5）：

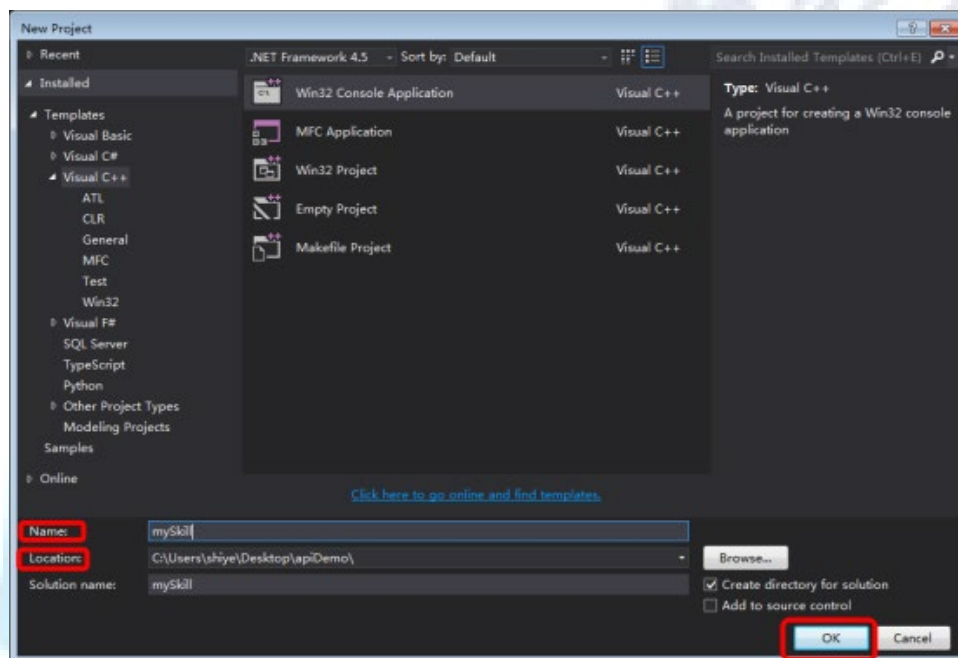


图 1-5 VS2013 创建新 project 窗口

- ✓ Name 项为 project 名称，即为自定义 skill 名，可以自行设置，示例中设为 mySkill；
- ✓ Location 项为 project 路径，可以自行设置；

- ✓ 勾选 Create directory for solution 项（表示在用户 project 路径下创建新的文件夹）；
- ✓ 单击 Win32 Console Application，并 OK。弹出图 1-6 界面：

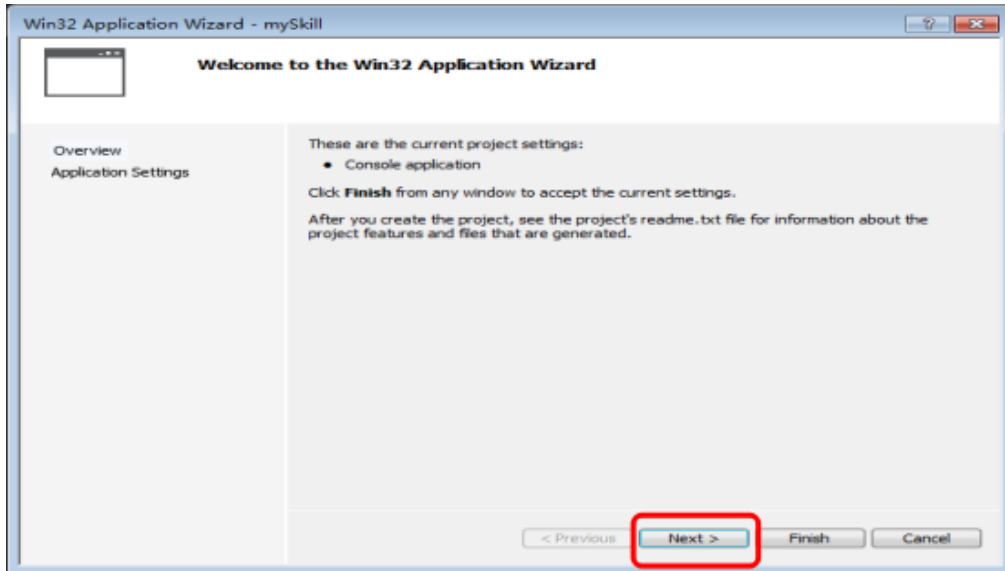


图 1-6

- 3) 直接单击 Next，弹出 1-7 界面：

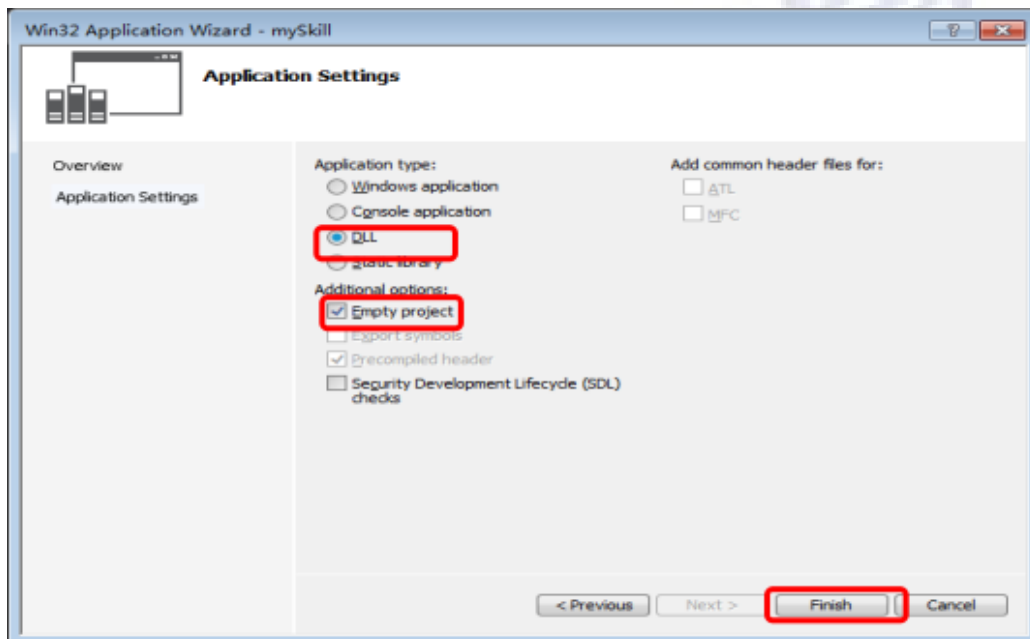


图 1-7

在此界面中选择 DLL 项和 Empty project 项，其他均不选，完成后单击 Finish。

- 4) 进入到 VS2013 主界面（图 1-8）：

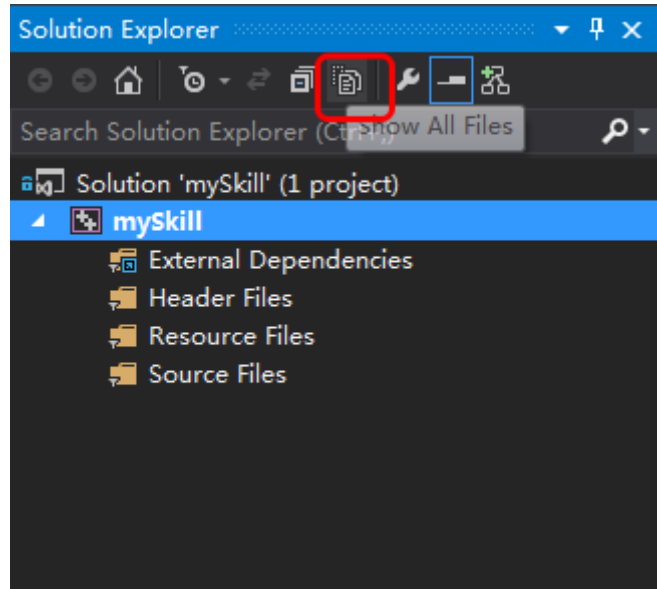


图 1-8

单击 Show All Files 按钮，项目创建完成。

4. 在新建项目中加载官方提供的 Utils 工具包

1) 进入 到用户 project 路径，示例中为：

C:\Users\shiye\Desktop\apiDemo\mySkill\mySkill

在此路径下创建 src 文件夹，并将官方提供的 utils 工具包拷贝到此。

2) 进入 VS2013 用户 project 主界面，鼠标右键单击项目名->Add->Existing

Items... 如图 1-9

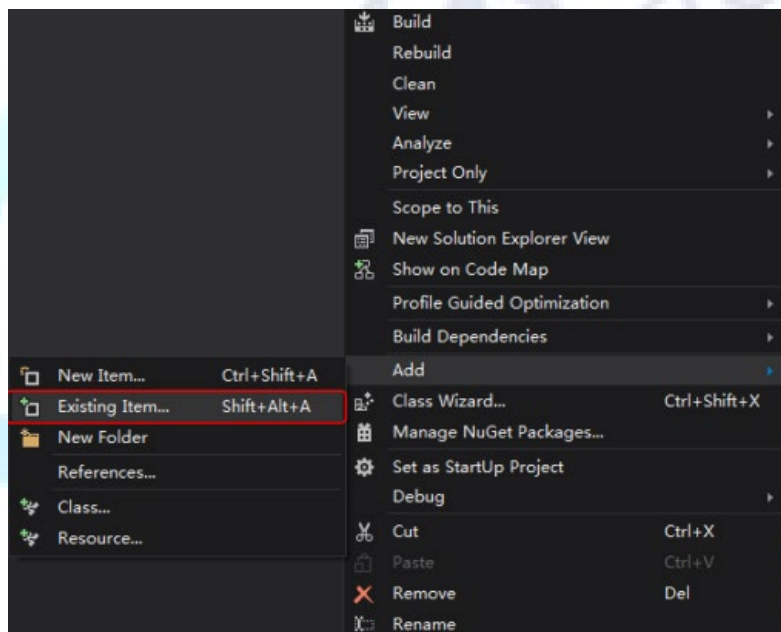


图 1-9

3) 弹出图 1-10 界面：选择 src 目录，点击 Add；

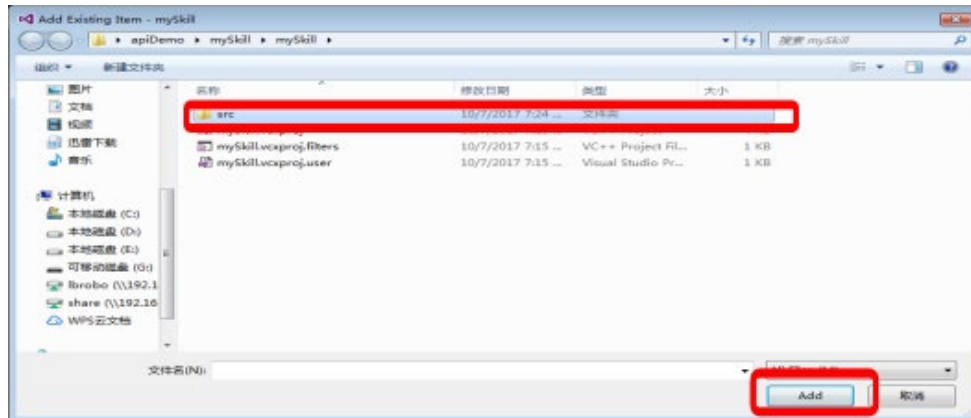


图 1-10 添加 utils 路径

4) 弹出图 1-11 界面，再选择 utils 目录，

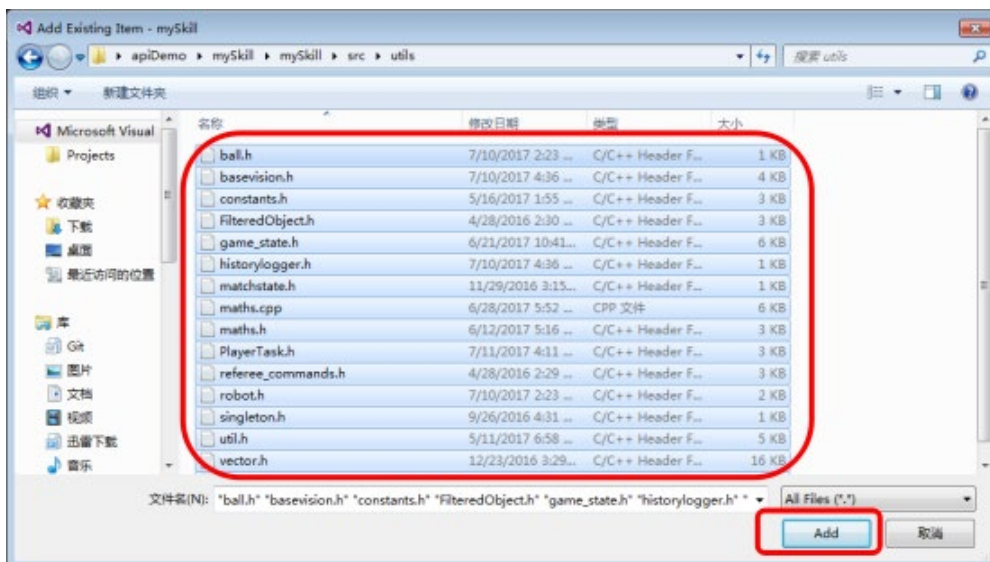


图 1-11 添加 utils 文件

如图 1-11 全选 utils 所含文件，并 Add;至此，utils 工具包文件加载完毕。

5. 在新建项目中加载官方提供的 worldmodel_lib.lib 库文件

1) 进入用户 project 路径，示例中为

C:\Users\shiye\Desktop\apiDemo\mySkill

将包含 worldmodel_lib.lib 的 worldmodel_lib 文件夹拷贝到此。

2) 进入 vs2013 用户 project 用户主界面，鼠标右键单击项目名->properties，

弹出图 1-12 界面：

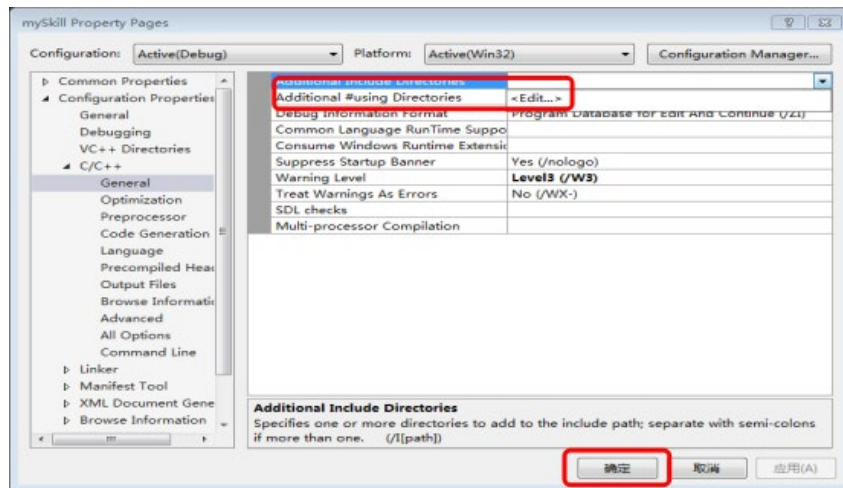


图 1-12

- 3) 选择 c/c++->General->Additional Include Directories, 单击 <Edit...>。弹出图 1-13 窗口:

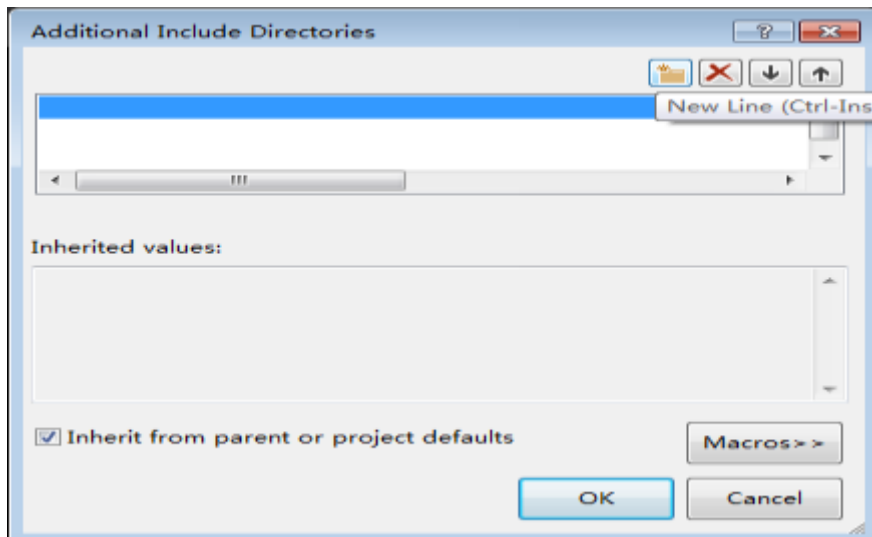


图 1-13

- 4) 单击 New Line 按钮, 将之前拷贝到用户 project 路径中的 worldmodel_lib 文件夹选中并指定到项目中。
- 5) 在 properties 窗口中, 选择 Linker->General->Additional Library Directories, 将 worldmodel_lib 文件夹选中并指定到项目中。如图 1-14

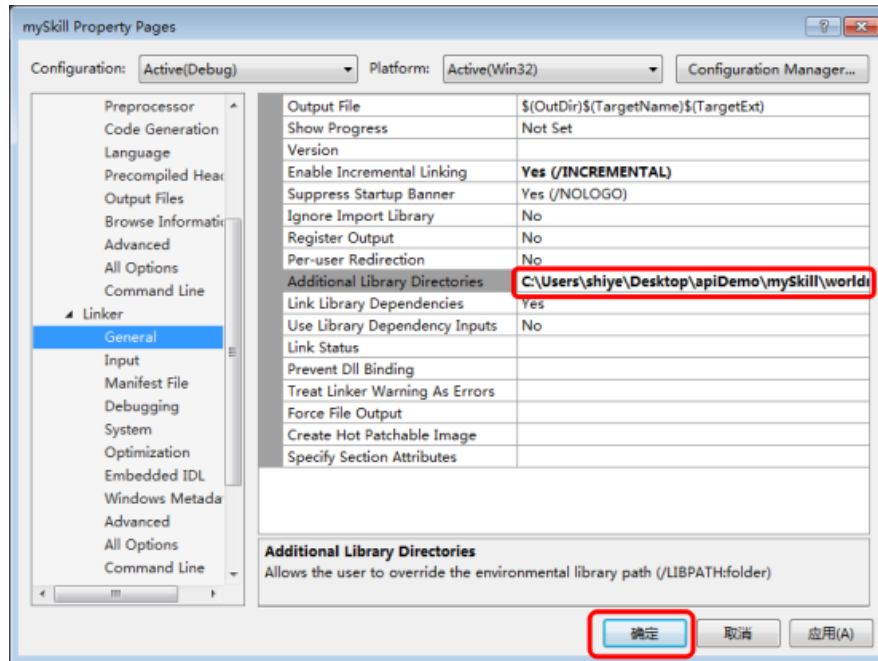


图 1-14

- 6) 在 properties 窗口中，选择 Linker->Input->Additional Dependencies，单击<Edit...>，如图 1-15

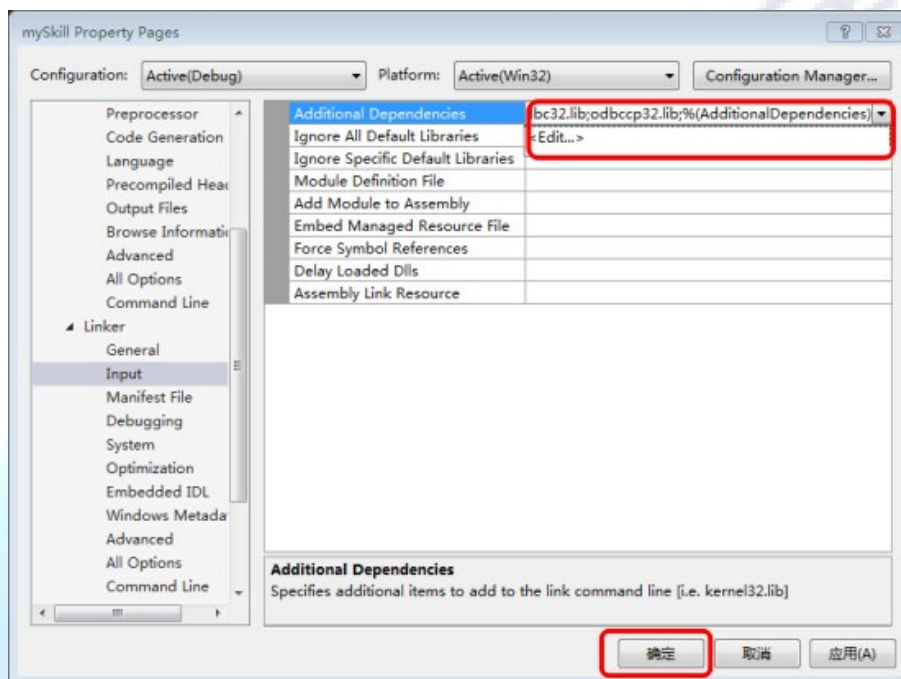


图 1-15

- 7) 将 worldmodel_lib.lib 文件全名 copy 到弹出的窗口中并点击“OK”，如图 1-16

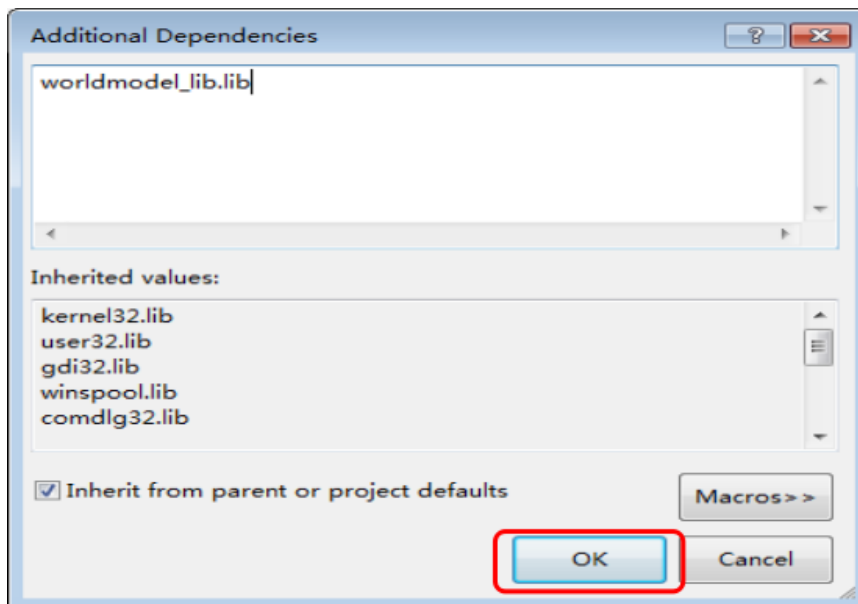


图 1-16

至此，SOM-V3.4 之 C++开发环境搭建完成。

说明：用户如果希望能进一步掌握 C++的语法系统，可以参考查阅《C++ Primer》。该计算机语言在网上有丰富的资料，用户可以自行学习。（此内容已超出 SOM-V3.4 二次开发手册范围，在此不做详细说明）

1.3 适配硬件

SOM-V3.4 可适配的机器人型号如下：

系列	型号	上市时间
ES	ES-SS01-H30	2015.10.01
ES	ES-NAA1	2016.06.15
ES	ES-NAB1	2016.11.30
ES	ES-NAC1	2017.8.10

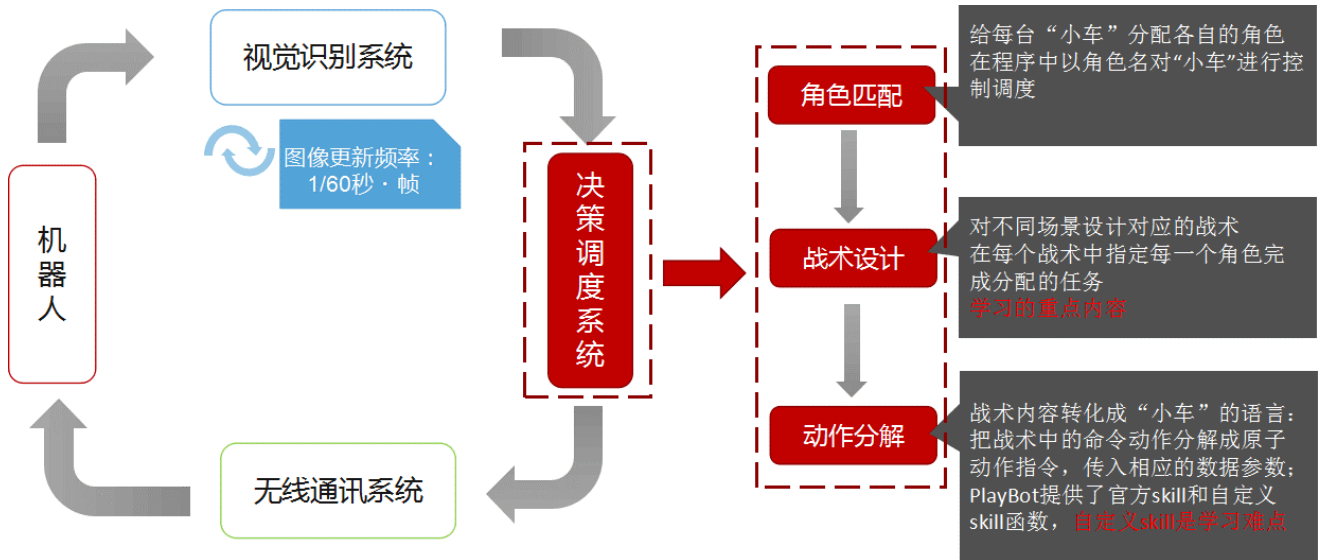
1.4 名词解释

LUA	一种脚本语言，可以很容易的被 c/c++调用，也可以调用 c/c++的函数，并且更容易理解和维护
SOM	乐博小型足球机器人操作管理系统

SRB	乐博小型足球机器人仿真模拟裁判盒软件
SV	乐博小型足球机器人视觉系统
play	把对多台机器人下达的一次运动决策称为一个 play
task	把一次运动决策所分配到各个机器人所需各自执行的任务称为 task
skill	将机器人所能完成的一个单项动作称为 skill
worldmodel _lib.lib	官方提供的静态库
utils	官方提供的工具包

2 系统框架

2.1 ES 系列机器人整体系统框架



从图 2-1 可以看出，决策调度是机器人的大脑，是 SOM 软件系统的核心。

2.2 SOM 软件系统架构

SOM 在软件系统结构上分为前端、后端，如下图 2-2 所示。

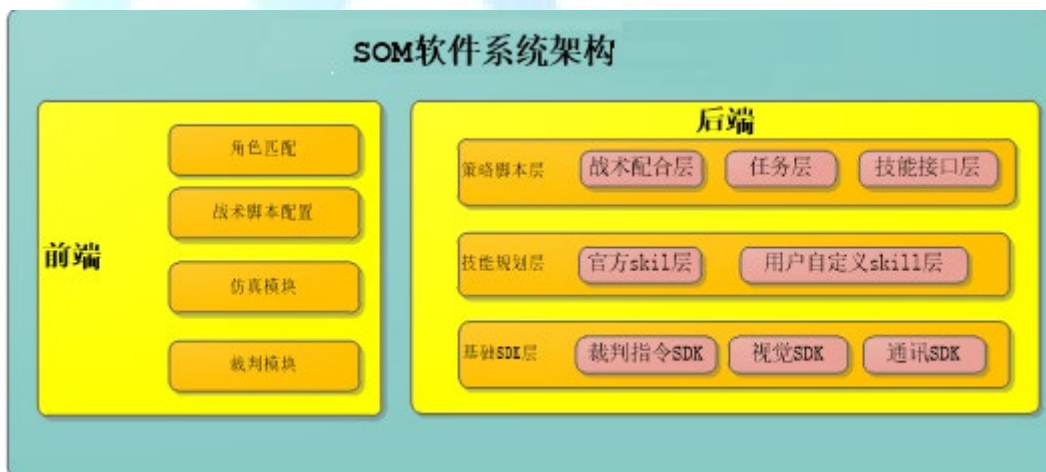


图 2-2 SOM 软件系统架构

2.2.1 前端

主要提供界面操作的主要功能，包括角色匹配、战术脚本配置、模拟仿真、裁判（此处裁判功能只是为了用户在本机使用时方便操作，如果正式比赛时，需使用单独的裁判盒软件 SRB 才能控制两支队伍正常进行比赛）

2.2.2 后端

主要实现决策调度，分为策略脚本层、技能规划层、基础 SDK 层。

SOM V3.4 软件目前支持策略脚本层、技能规划层的二次开发；策略脚本层使用 LUA 脚本语言实现；技能规划层和基础 SDK 层使用 C++ 语言实现。

3 策略脚本层（LUA）框架

策略脚本层，分为三层：战术配合层 (play)、任务层 (task)、技能接口层 (简称技能层 skill)

- 战术配合层：将各种任务和角色通过战术设计组织起来
- 任务层：对技能层实现上层封装，即任务层会调用技能层，并将 task 表参数传递给技能层。
- 技能层：实现机器人的单项动作。

3.1 战术配合层（play）

3.1.1 定义

- 我们把对多台机器人下达的一次运动决策称为一个 play，将各种任务和角色通过战术策略设计组织起来，以一个 LUA 文件形式存在。
- **一次运动决策的执行需要在 1/60s 完成**。如图 2-1 中所示，视觉系统的图像更新频率是 60FPS，即 1 秒钟处理 60 帧图像，那么也就意味着，决策调度模块，需要在 1 帧的时间内即 1/60s，完成一次运动决策。
- 通过 SOM 软件前端的“脚本导入”功能导入的 play 脚本都放置在软件安装目录的如下路径：



图 3-1 play 脚本目录路径

3.1.2 说明

play 脚本分为 Test、Nor、Ref 三类



图 3-2 play 的分类

3.1.2.1 Test 脚本

在没有转为正式脚本前的脚本，都属于**测试脚本**，系统会自动存放在 Test 目录下。

3.1.2.2 Nor 脚本

在没有裁判盒指令干预下执行的正式脚本，称为**正常脚本**，当用户在选择转正式脚本时，如果选择的战术类型为“Normal”，系统会自动存放在 Nor 目录下。

例如：正常的攻防转换脚本，一般可以作为 Nor 脚本。

3.1.2.3 Ref 脚本

在裁判盒指令干预下执行的正式脚本，称为**裁判脚本**，当用户在选择转正式脚本时，如果选择的战术类型为裁判指令对应的战术类型，系统会自动存放到 Ref 目录下。

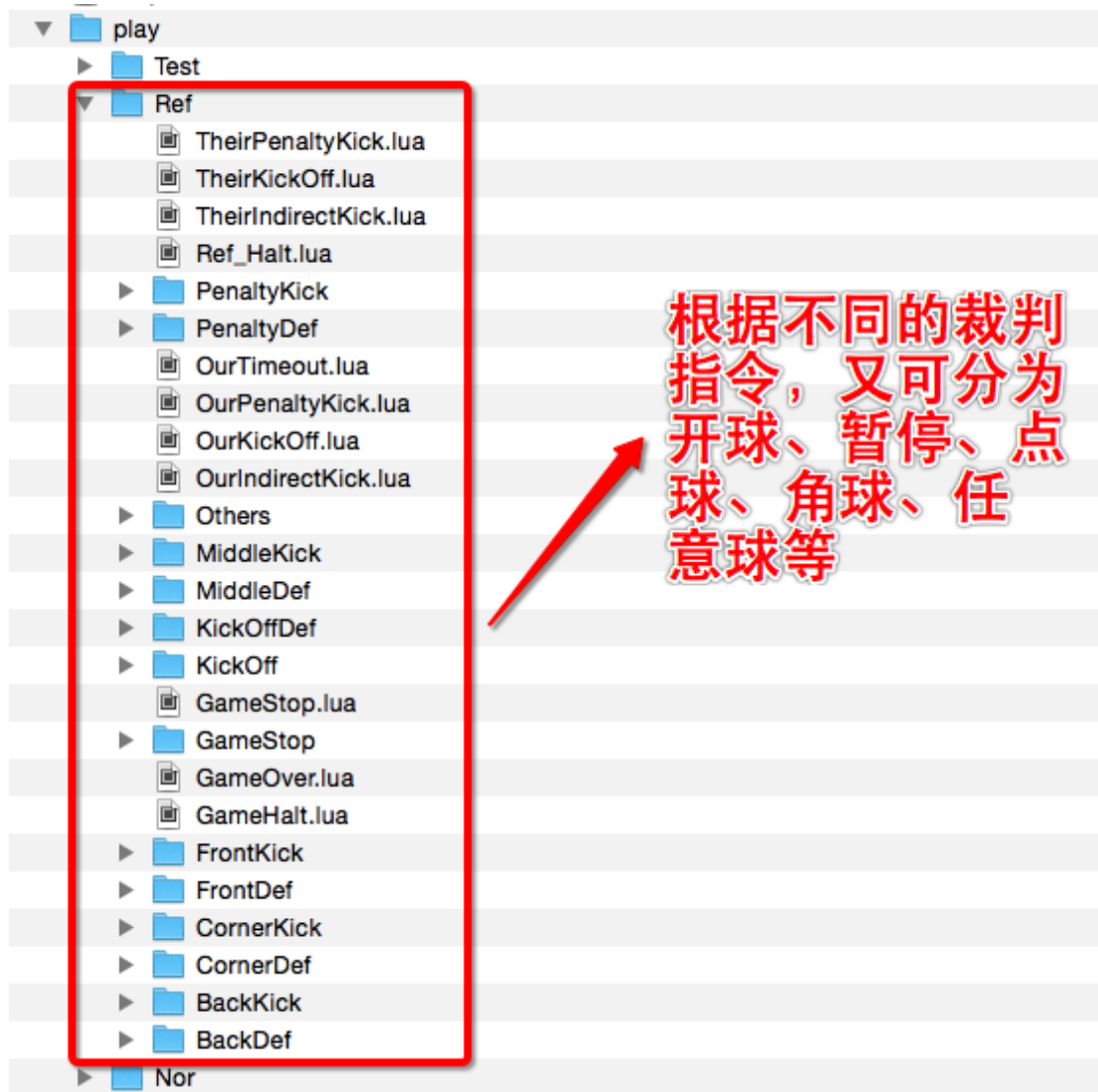


图 3-3 Ref 脚本目录结构

3.2 任务层 (task)

3.2.1 定义

- 我们把一次运动决策所分配到各个机器人所需各自执行的任务称为 task。每个 task 以函数的形式存在，函数定义在 task.lua 文件中。
- 官方提供的 task.lua 在如下路径中：



图 3-4 官方提供的 task.lua 路径

- 官方提供的 task.lua 中的各函数说明详见[《附件 1: task 函数说明》](#)

3.2.2 说明

任务层函数分为两大类：

- 官方 task 函数
 - 1) 官方 task 函数调用官方提供的技能函数；
 - 2) 官方 task 函数主要有 GetBall()、PassBall()、ReceiveBall()、Shoot()、Goalie() 等 13 个；函数说明详见[《附件 1: task 函数说明》](#)中 6.1.7-6.1.19
- 自定义 task 函数
 - 1) 自定义 task 函数，使用官方 task.lua 提供的标准入口函数，通过加载用户自己编写的技能 dll 来实现。
 - 2) 自定义 task 函数有 KickerTask()、ReceiverTask()、TierTask()、DefenderTask()、MiddleTask()、GoalieTask()；函数说明详见[《附件 1: task 函数说明》](#)中 6.1.1-6.1.6

- 3) 自定义 task 函数只能给指定的角色使用，如 KickerTask 对应 Kicker(前锋)，ReceiverTask 对应 Receiver(中场)、Tier 对应 Tier(后卫)、GoalieTask 对应 Goalie(守门员)等。

3.3 技能层 (skill)

3.3.1 定义

- 我们将机器人所能完成的一个单项动作称为 skill,每个 skill 以函数的形式存在，函数定义在 skill lua 文件中。
- 官方提供的 skill lua 文件在如下路径中：

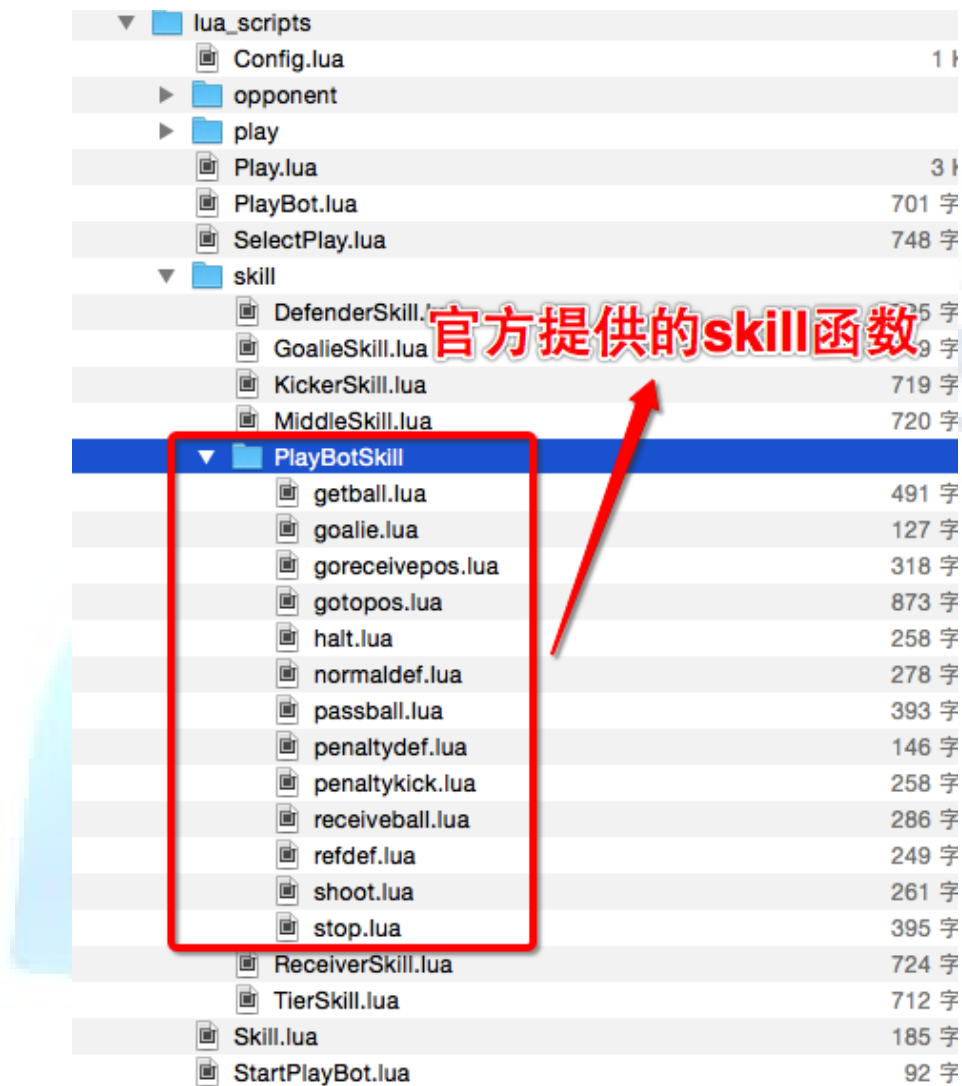


图 3-5 官方提供的 skill 所在路径

3.3.2 说明

技能层函数分为两大类：

- 官方 skill 函数

- 1) 官方提供的 skill 函数调用官方提供的 cpp
- 2) 此类 skill 在 lua_scripts\skill\PlayBotSkill 路径下，以独立的 lua 文件形式存在

- 用户自定义 skill 函数

- 1) 用户自定义 skill 需要用户使用 C++编写，最终以 dll 的方式加载到如下目录：

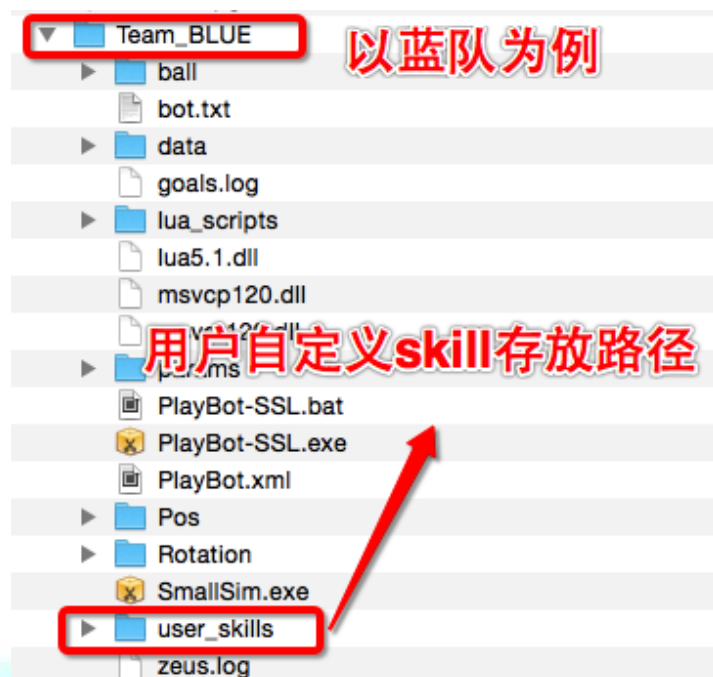


图 3-6 用户自定义 skill 加载路径

- 2) 使用时，由自定义 task 函数加载用户自己编写的技能 dll 来实现。

3.4 基础函数

用户可以在 LUA 脚本中直接调用官方提供的基础函数获取球场信息或判断球场信息。基础函数说明详见[《附件 2：基础函数说明》](#)。

用例：

```
if CIsBallKick("Receiver") then ... end
```

判断“Receiver”是否将球踢出。

4 决策规划层（C++）扩展

4.1 扩展内容

SOM-V3.4 软件允许用户通过官方提供的 `worldmodel_lib` 静态库获得球场信息，并在 `player_plan` 函数中对 `PlayerTask` 对象实现自定义技能，扩展步骤如下：

1. 参照 [1.2.3](#) 中的步骤搭建 C++ 开发环境，根据 [4.4](#) 中的框架使用 C++ 编写自定义技能
2. 将编写好的技能编译生成 `dll` 文件，并将 `dll` 文件 `copy` 到 `user_skills` 目录下
3. 根据 [5.2](#) 中的示例编写 `LUA` 脚本，通过自定义 `task` 函数调用用户自定义的技能

4.2 静态库

SOM-V3.4 软件为用户提供了 `worldmodel_lib.lib` 静态库，静态库为用户提供了所需要获得的球场信息接口（如小球，我方球员机器人，对方球员机器人相关信息）。静态库的加载方法请[详见 1.2.3 章节](#)。

4.3 工具包

SOM-V3.4 软件为用户提供了 `utils` 工具包，包内为用户扩展 `skill` 提供了所需要用到的 C++ 算法函数接口、常量参数接口等。工具包的加载方法请[详见 1.2.3 章节](#)。

4.4 自定义技能 C++ 框架

```
extern "C"__declspec(dllexport) PlayerTask player_plan(const
WorldModel* model, int robot_id);//绿色部分为声明函数player_plan

PlayerTask player_plan(const WorldModel* model, int robot_id){
    PlayerTask task;

    return task;

//紫色部分为生成一个 PlayerTask 对象，通过对象实现 skill 功能，
```

```
//并将其返回给调用者
} //红色部分为定义函数体 player_plan
```

注意：用户必须严格按照框架中的方法声明 `player_plan` 函数，并定义函数。

4.5 用户技能 dll 放置目录

在 SOM-V3.4 之 c++开发环境中编写程序，生成的 dll 需要放置到指定文件夹位置。如图

4-1

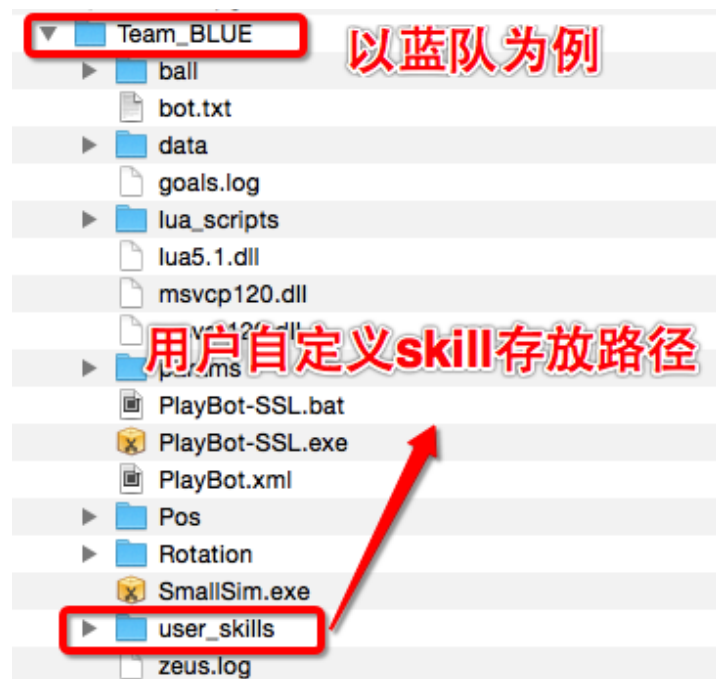


图 4-1 用户自定义技能 dll 存放路径

其中 Team_BLUE (or Team_YELLOW) 是队伍策略脚本目录，需要用户根据需求决定。如 dll 给蓝队使用，就放置在蓝队的 user_skills 目录下。

5 实战案例

5.1 官方 task 实战场景

5.1.1 战术设计

编写一个前场间接任意球 play，实现一传一射，均使用官方提供的 task

战术框架：

```
gPlayTable.CreatePlay{ --红色部分为战术框架主结构
firstState = "",
[] = { --紫色部分为状态框架
    switch = function() --蓝色部分为状态跳转函数
        if ... then
            return ...
        end,
        Role = task --绿色部分为角色、任务分配
    },
[] = {
    switch = function()
        if ... then
            return ...
        end,
        Role = task
    },
[] = {
    switch = function()
        if ... then
            return ..
        end,
        Role = task
    },
name = "" --此处为脚本名
}
```

如 [1.2.2](#) 所述，用户编写战术脚本必须严格按照此框架实现。

5.1.2 战术分解

5.1.2.1 第一步：拿球（“GetBall”）

1) 助攻任务：让中场足球机器人去拿球，小球可以是场上任意位置

```
Receiver = task.GetBall("Receiver", "Receiver")
```

--Receiver 为中场角色名

--GetBall("Receiver", "Receiver") 为指定“Receiver”角色朝向球门拿球，

--GetBall 函数说明[详见 6.1.7](#)

2) 射手任务：前锋去接球点，接球点的位置是按照场上球的位置做相应逻辑获得的

```
Kicker = task.GoRecePos("Kicker")
```

--Kicker 为中场角色名

--GoRecePos 为指定“Kicker”角色去接球点

--GoRecePos 函数说明[详见 6.1.9](#)

3) 跳转条件：当接球者离球的距离小于 30mm 时，状态跳转到第二步“PassBall”

```
if CBall2RoleDist("Receiver") < 30 then
```

```
return "PassBall"
```

```
End
```

--CBall2RoleDist("Receiver") 函数返回“Receiver”到球的距离

--CBall2RoleDist 函数说明[详见 6.2.1.4](#)

5.1.2.2 第二步：传球（“PassBall”）

1) 助攻任务：让中场足球机器人传球给前锋

```
Receiver = task.PassBall("Receiver", "Kicker")
```

--Receiver 为传球角色名

--Kicker 为接球角色名

--PassBall 函数说明[详见 6.1.12](#)

2) 射手任务：前锋去接球点，接球点的位置是按照场上球的位置做相应逻辑获得的

```
Kicker = task.GoRecePos("Kicker")
```

```
--Kicker 为中场角色名
--GoRecePos 为指定“Kicker”角色去接球点
--GoRecePos 函数说明详见 6.1.9
```

3) 跳转条件：当传球者将球踢出，状态跳转到第三步“Shoot”

```
if ClsBallKick("Receiver") then

    return "Shoot"

end

--ClsBallKick("Receiver")函数返回“Receiver”是否将球踢出的判断
--ClsBallKick 函数说明详见 6.2.1.10
```

5.1.2.3 第三步：射门 (“Shoot”)

1) 助攻任务：中场球员进入防守任务

```
Receiver = task.RefDef("Receiver")

--Receiver 为防守角色名
--RefDef 函数说明详见 6.1.16
```

2) 射手任务：前锋射门

```
Kicker = task.Shoot("Kicker")

--Kicker 为射门角色名
--Shoot 函数说明详见 6.1.15
```

3) 跳转条件：当射门者将球踢出，状态结束

```
if ClsBallKick("Kicker") then

    return "finish"

End

--ClsBallKick("Kicker")函数返回“Kicker”是否将球踢出的判断
--ClsBallKick 函数说明详见 6.2.1.10
```

5.1.3 完整示例 play 代码

示例战术脚本名为 Ref_passAndShoot.lua

```
gPlayTable.CreatePlay{
```

```
firstState = "GetBall",

["GetBall"] = {
    switch = function()
        if CBall2RoleDist("Receiver") < 30 then
            return "PassBall"
        end
    end,
    Kicker = task.GoRecePos("Kicker"),
    Receiver = task.GetBall("Receiver", "Receiver"),
    Goalie = task.Goalie()
},

["PassBall"] = {
    switch = function()
        if CIsBallKick("Receiver") then
            return "Shoot"
        end
    end,
    Kicker = task.GoRecePos("Kicker"),
    Receiver = task.PassBall("Receiver", "Kicker"),
    Goalie = task.Goalie()
},

["Shoot"] = {
    switch = function()
        if CIsBallKick("Kicker") then
            return "finish"
        end
    end
}
```

```

    end,

    Kicker = task.Shoot("Kicker"),

    Receiver = task.RefDef("Receiver"),

    Goalie = task.Goalie()

},

name = "Ref_passAndShoot "

}

```

5.2 自定义 task 实战场景

5.2.1 战术设计

编写一个防守战术脚本，其中 Kicker 调用的是官方 task；Receiver 调用的是用户自定义 task。在用户自定义 task 中导入的 dll 实现了根据小球在左中右的位置计算不同的防守站位及朝向角的技能。

战术框架：

```

gPlayTable.CreatePlay{ --红色部分为战术框架主机构
    firstState = "doDef",

    switch = function() --深蓝部分为状态跳转函数
        return "doDef"
    end,

    ["doDef"] = { --紫色部分为状态框架
        Role = task --绿色部分为角色、任务分配
    },

    name = "" --此处为脚本名
}

```


5.2.2 战术分解

1) 防守任务 1: 前锋防守

```
Kicker = task.RefDef("Kicker")
```

--"Kicker"角色执行防守任务

2) 防守任务 2: 中场防守

```
Receive = task.ReceiverTask("def")
```

--ReceiverTask()函数调用用户自定义的task技能;

--"def"技能由C++编写设计, 生成的def.dll 放置在 user_skills 中

--每一个角色都有固定的自定义task函数

--如 KickerTask(), TierTask(), GoalieTask() 等

--说明: def.dll 的源代码, 用户可以在官方提供的 demo 中查看

3) 跳转条件: 一直执行防守状态

```
switch = function()
```

```
    return "doDef"
```

```
end
```

5.2.3 完整 play 代码

示例战术脚本名为 Ref_KickDef.LUA

```
gPlayTable.CreatePlay{  
firstState = "doCornerDef",  
switch = function()  
    return "doDef"  
end,  
["doDef"] = {  
    Kicker = task.RefDef("Kicker"),  
    Receive = task.ReceiverTask("def"),  
    Goalie = task.Goalie()  
},  
  
name = "Ref_KickDef"
```

}



6 附件

6.1 附件 1: task 函数说明

说明:

- 6.1.1-6.1.6 函数为官方提供的自定义 task 函数，传入用户编写的 dll 技能，实现扩展
- 每个自定义 task 函数指定给对应的角色使用，如 KickerTask 对应 Kicker（前锋）、ReceiverTask 对应 Receiver（中场）等。
- 使用方法示例:

Kicker = KickerTask(“dll 名称”, pos_, dir_, kickflat_, kp_, cp_)

6.1.1 KickerTask

函数名	KickerTask(name_, pos_, dir_, kickflat_, kp_, cp_)		
实现功能	官方提供的自定义 task 函数，指定 Kicker 角色使用		
输入参数	参数名	参数类型	参数说明
	name_	string	用户编写 C++ 编译生成的动态库名
	posX	number	目标点 x 轴坐标 (单位: cm)
	posY	number	目标点 y 轴坐标 (单位: cm)
	dir_	number	和目标点连线与 x 轴的夹角, (单位: 角度)
	kickflat_	bool	是否平射
	kp_	number	踢球平射力度
	cp_	number	踢球挑射力度
返回参数			

6.1.2 ReceiverTask

函数名	ReceiverTask(name_, pos_, dir_, kickflat_, kp_, cp_)		
实现功能	官方提供的自定义 task 函数，指定 Receiver 角色使用		

	参数名	参数类型	参数说明
输入参数	name_	string	用户编写 C++编译生成的动态库名
	posX	number	目标点 x 轴坐标 (单位: cm)
	posY	number	目标点 y 轴坐标 (单位: cm)
	dir_	number	和目标点连线与 x 轴的夹角, (单位: 角度)
	kickflat_	bool	是否平射
	kp_	number	踢球平射力度
	cp_	number	踢球挑射力度
返回参数			

6.1.3 TierTask

函数名	TierTask(name_,pos_,dir_,kickflat_,kp_,cp_)		
实现功能	官方提供的自定义 task 函数, 指定 Tier 角色使用		
输入参数	参数名	参数类型	参数说明
	name_	string	用户编写 C++编译生成的动态库名
	posX	number	目标点 x 轴坐标 (单位: cm)
	posY	number	目标点 y 轴坐标 (单位: cm)
	dir_	number	和目标点连线与 x 轴的夹角, (单位: 角度)
	kickflat_	bool	是否平射
	kp_	number	踢球平射力度
	cp_	number	踢球挑射力度
返回参数			

6.1.4 DefenderTask

函数名	DefenderTask(name_,pos_,dir_,kickflat_,kp_,cp_)		
实现功能	官方提供的自定义 task 函数, 指定 Defender 角色使用		
输入参数	参数名	参数类型	参数说明
	name_	string	用户编写 C++编译生成的动态库名
	posX	number	目标点 x 轴坐标 (单位: cm)

			cm)
	posY	number	目标点 Y 轴坐标 (单位: cm)
	dir_	number	和目标点连线与 x 轴的夹角, (单位: 角度)
	kickflat_	bool	是否平射
	kp_	number	踢球平射力度
	cp_	number	踢球挑射力度
返回参数			

6.1.5 MiddleTask

函数名	MiddleTask(name_,pos_,dir_,kickflat_,kp_,cp_)		
实现功能	官方提供的自定义 task 函数, 指定 Middle 角色使用		
输入参数	参数名	参数类型	参数说明
	name_	string	用户编写 C++编译生成的动态库名
	posX	number	目标点 X 轴坐标 (单位: cm)
	posY	number	目标点 Y 轴坐标 (单位: cm)
	dir_	number	和目标点连线与 x 轴的夹角, (单位: 角度)
	kickflat_	bool	是否平射
	kp_	number	踢球平射力度
	cp_	number	踢球挑射力度
返回参数			

6.1.6 GoalieTask

函数名	GoalieTask(name_,pos_,dir_,kickflat_,kp_,cp_)		
实现功能	官方提供的自定义 task 函数, 指定 Goalie 角色使用		
输入参数	参数名	参数类型	参数说明
	name_	string	用户编写 C++编译生成的动态库名
	posX	number	目标点 X 轴坐标 (单位: cm)
	posY	number	目标点 Y 轴坐标 (单位: cm)
	dir_	number	和目标点连线与 x 轴的

			夹角，（单位：角度）
	kickflat_	bool	是否平射
	kp_	number	踢球平射力度
	cp_	number	踢球挑射力度
返回参数			

说明:

- 6.1.7-6.1.19 函数为官方 task 函数，调用官方提供的技能函数；
- 每一个官方 task 函数完成一项技能，并且可以指派给所有的角色使用。

6.1.7 GetBall

函数名	GetBall(runner_role, receive)		
实现功能	实现朝向某个角色拿球，如果第一个和第二个参数相同，则为朝向球门拿球。		
输入参数	参数名	参数类型	参数说明
	runner_role	string	第一个参数是：执行者的角色名
	receive	string	第二个参数是：朝向球员的角色名
返回参数			

6.1.8 Goalie

函数名	Goalie		
实现功能	实现守门，根据场上的球状态（进攻或者防守），做相应的守门策略		
输入参数	参数名	参数类型	参数说明
	无		
返回参数			

6.1.9 GoRecePos

函数名	GoRecePos(role_name_)		
实现功能	去接球点，接球点的位置是按照场上球的位置做相应逻辑获得的		

输入参数	参数名	参数类型	参数说明
	role_name_	string	参数为执行者角色名
返回参数			

6.1.10 RobotHalt

函数名	RobotHalt(role_name_)		
实现功能	急停，立即停止，车保持原地不动		
输入参数	参数名	参数类型	参数说明
	role_name_	string	参数为执行者角色名
返回参数			

6.1.11 NormalDef

函数名	NormalDef(role_name_)		
实现功能	普通状态下防守。防守是按照球的位置决定防守点。		
输入参数	参数名	参数类型	参数说明
	role_name_	string	参数为执行者角色名
返回参数			

6.1.12 PassBall

函数名	PassBall(runner_role_,receive_)		
实现功能	朝向某个角色传球		
输入参数	参数名	参数类型	参数说明
	runner_role_	string	参数为执行者角色
	receive_	string	参数为被传球者角色名
返回参数			

6.1.13 ReceiveBall

函数名	ReceiveBall(role_name_)		
实现功能	接球，接球点的位置是按照场上球的位置做相应的逻辑获得的		
输入参数	参数名	参数类型	参数说明
	role_name_	string	参数为执行者角色名

返回参数			
------	--	--	--

6.1.14 Stop

函数名	Stop(role_name_,role_)		
实现功能	停止		
输入参数	参数名	参数类型	参数说明
	role_name_	string	参数为执行者角色名
	role_	number	分为 1、2、3、4、5、6 共计 6 种情况,根据不用角色停球位置不同。(1 代表前锋,离球 50cm; 2 代表前腰;3 代表中场; 4 代表后腰;5 代表后卫; 6 代表守门员)
返回参数			

6.1.15 Shoot

函数名	Shoot(role_name_)		
实现功能	射门,朝向位置是按照场上球门的位置做相应的逻辑获得的		
输入参数	参数名	参数类型	参数说明
	role_name_	string	参数为执行者角色名
返回参数			

6.1.16 RefDef

函数名	RefDef(role_name_)		
实现功能	裁判盒状态下防守,接收到裁判指令后的防守策略		
输入参数	参数名	参数类型	参数说明
	role_name_	string	参数为执行者角色名
返回参数			

6.1.17 GotoPos

函数名	GotoPos(role_name_,x_,y_,dir_)		
实现功能	去某个点，球员跑位到某点后，并朝向某一角度		
输入参数	参数名	参数类型	参数说明
	role_name_	string	参数为执行者角色名
	x_	number	点的x轴坐标(单位:cm)
	y_	number	点的y轴坐标(单位:cm)
	dir_	number	机器人朝向，(单位:角度)
返回参数			

6.1.18 PenaltyDef

函数名	PenaltyDef		
实现功能	点球防守，在接收到裁判盒点球命令后执行防守		
输入参数	参数名	参数类型	参数说明
	无		
返回参数			

6.1.19 PenaltyKick

函数名	PenaltyKick(role_name_)		
实现功能	罚点球，在接收到罚点球命令后执行点球		
输入参数	参数名	参数类型	参数说明
	role_name_	string	参数为执行者角色名
返回参数			

6.2 附件 2：基础函数说明

以下函数为 **LUA** 层可以直接调用的函数

6.2.1 比赛信息：我方球员

6.2.1.1 COurRole_x

函数名	COurRole_x(role)		
实现功能	官方提供的 C++ 函数，获取我方球员 x 轴坐标值		
输入参数	参数名	参数类型	参数说明
	role	string	角色名
返回参数	pos_x	number	x 轴坐标值

6.2.1.2 COurRole_y

函数名	COurRole_y(role)		
实现功能	官方提供的 C++ 函数，获取我方球员 y 轴坐标值		
输入参数	参数名	参数类型	参数说明
	role	string	角色名
返回参数	pos_y	number	y 轴坐标值

6.2.1.3 COurRoleDir

函数名	COurRoleDir(role)		
实现功能	官方提供的 C++ 函数，获取我方球员朝向		
输入参数	参数名	参数类型	参数说明
	role	string	角色名
返回参数	dir	number	角色朝向（与 X 轴正向夹角）（单位：角度）

6.2.1.4 CBall2RoleDist

函数名	CBall2RoleDist(role)		
实现功能	官方提供的 C++ 函数，到球距离，获取我方球员到球的直线距离。		
输入参数	参数名	参数类型	参数说明
	role	string	角色名
返回参数	dist	number	到球的距离 (单位：cm)

6.2.1.5 CRole2BallDir

函数名	CRole2BallDir (role)		
实现功能	官方提供的 C++ 函数，到球朝向，获取我方球员到球的朝向角度。		
输入参数	参数名	参数类型	参数说明
	role	string	角色名
返回参数	dir	number	与球的夹角(单位:角度)

6.2.1.6 COurRole2RoleDist

函数名	COurRole2RoleDist (role, role)		
实现功能	官方提供的 C++ 函数，到我方某个角色距离，获取 2 者之间直线距离		
输入参数	参数名	参数类型	参数说明
	role	string	参数 1 为当前球员
	role	string	参数 2 为目标球员
返回参数	dist	number	两名角色间距离(单位:cm)

6.2.1.7 CRole2TargetDist

函数名	CRole2TargetDist (role)		
实现功能	官方提供的 C++ 函数，离目标点的距离，角色球员距离其目标点的距离，目标点由 task 任务决定		
输入参数	参数名	参数类型	参数说明
	role	string	角色名
返回参数	dist	number	角色到目标点的距离(单位: cm)

6.2.1.8 COurRole2RoleDir

函数名	COurRole2RoleDir (role, role)		
实现功能	官方提供的 C++ 函数，与我方某个角色朝向，获取 2 者之间朝向角		
输入参数	参数名	参数类型	参数说明
	role	string	参数 1 为当前球员

	role	string	参数 2 为目标球员
返回参数	dir	number	两名角色间夹角（单位：角度）

6.2.1.9 CRole2OppGoalDir

函数名	CRole2OppGoalDir (role)		
实现功能	官方提供的 C++ 函数，到对方球门朝向角，获取角色到对方球门朝向角，其中球门位置为球门中心点坐标		
输入参数	参数名	参数类型	参数说明
	role	string	角色名
返回参数	dir	number	角色到对方球门朝向角（单位：角度）

6.2.1.10 CIsBallKick

函数名	CIsBallKick (role)		
实现功能	官方提供的 C++ 函数，是否踢球，获取角色是否进行了踢球		
输入参数	参数名	参数类型	参数说明
	role	string	角色名
返回参数		bool	true 踢球，false 未踢球

6.2.1.11 CIsGetBall

函数名	CIsGetBall (role)		
实现功能	官方提供的 C++ 函数，是否拿球，获取角色是否拿球		
输入参数	参数名	参数类型	参数说明
	role	string	角色名
返回参数		bool	true 拿球，false 未拿球

6.2.2 比赛信息：敌方球员

6.2.2.1 CGetOppNums

函数名	CGetOppNums		
实现功能	官方提供的 C++ 函数，获取敌方所有上场车号，以表类型返回		
输入参数	参数名	参数类型	参数说明
返回参数		table 类型	上场车号数 例如：如果上场 2 号、5 号、12 号，返回数组为 [2, 5, 12]

注：CGetOppNums 返回的 table 存储格式是 { [0]="n1", [1]="n2", [2]="n3" }，存储顺序是随机的；其中 "n1", "n2", "n3" 表示返回的车号，车号是 string 类型。在实际应用中，我们需要用 for...in pairs(table) do... 的方式遍历 table 并找到场上敌方车号。

6.2.2.2 COppNum_x

函数名	COppNum_x(id)		
实现功能	官方提供的 C++ 函数，根据敌方球员车号获取敌方球员 x 轴坐标值		
输入参数	参数名	参数类型	参数说明
	id	int	敌方球员车号（取值范围 1-12）
返回参数	pos_x	number	x 轴坐标值

6.2.2.3 COppNum_y

函数名	COppNum_y(id)		
实现功能	官方提供的 C++ 函数，根据敌方球员车号获取敌方球员 y 轴坐标值		
输入参数	参数名	参数类型	参数说明
	id	int	敌方球员车号（取值范围 1-12）
返回参数	pos_y	number	y 轴坐标值

6.2.2.4 COppNumDir

函数名	COppNumDir(id)		
实现功能	官方提供的 C++ 函数，根据敌方球员车号获取敌方球员朝向。		
输入参数	参数名	参数类型	参数说明
	id	int	敌方球员车号（取值范围 1-12）
返回参数	dir	number	朝向（与 X 轴正向夹角） （单位：角度）

6.2.2.5 COppNum2BallDist

函数名	COppNum2BallDist(id)		
实现功能	官方提供的 C++ 函数，根据敌方球员车号获取敌方球员到球的直线距离。		
输入参数	参数名	参数类型	参数说明
	id	int	敌方球员车号（取值范围 1-12）
返回参数	dist	number	到球的距离（单位：cm）

6.2.2.6 COppNum2BallDir

函数名	COppNum2BallDir(id)		
实现功能	官方提供的 C++ 函数，根据敌方球员车号获取敌方球员到球的朝向角度。		
输入参数	参数名	参数类型	参数说明
	id	int	敌方球员车号（取值范围 1-12）
返回参数	dir	number	与球的朝向角（单位：角度）参见图 6.1

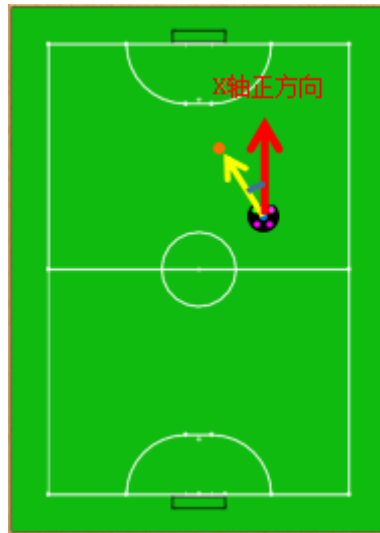


图 6.1 小球与车夹角关系图

6.2.2.7 COppIsBallKick

函数名	COppIsBallKick(id)		
实现功能	官方提供的 C++ 函数，根据敌方球员车号判断敌方球员是否踢球		
输入参数	参数名	参数类型	参数说明
	id	int	敌方球员车号（取值范围 1-12）
返回参数		bool	true 踢球，false 未踢球

6.2.2.8 COppIsGetBall

函数名	COppIsGetBall(id)		
实现功能	官方提供的 C++ 函数，根据敌方球员车号判断敌方球员是否拿球		
输入参数	参数名	参数类型	参数说明
	id	int	敌方球员车号（取值范围 1-12）
返回参数		bool	true 拿球，false 未拿球

6.2.3 比赛信息：球

6.2.3.1 CGetBallX

函数名	CGetBallX()		
实现功能	官方提供的 C++ 函数，x 位置，球的 x 坐标		
输入参数	参数名	参数类型	参数说明
	无		
返回参数	pos_x	number	x 轴坐标值

6.2.3.2 CGetBallY

函数名	CGetBallY()		
实现功能	官方提供的 C++ 函数，y 位置，球的 y 坐标		
输入参数	参数名	参数类型	参数说明
	无		
返回参数	pos_y	number	y 轴坐标值

6.2.3.3 CBall2PointDist

函数名	CBall2PointDist(our_goal_x,our_goal_y)		
实现功能	官方提供的 C++ 函数，离我方球门距离，球门为球门中心点位置		
输入参数	参数名	参数类型	参数说明
	our_goal_x	string	我方球门中心 x 轴坐标
	our_goal_y	string	我方球门中心 y 轴坐标
返回参数	dist	number	球与我方球门中心点的距离 (单位：cm)

6.2.3.4 CBall2PointDist

函数名	CBall2PointDist(opp_goal_x,opp_goal_y)		
实现功能	官方提供的 C++ 函数，离对方球门距离，球门为球门中心点位置		
输入参数	参数名	参数类型	参数说明
	opp_goal_x	string	对方球门中心 x 轴坐标

	opp_goal_y	string	对方球门中心 Y 轴坐标
返回参数	dist	number	球与对方球门中心点的距离 (单位: cm)

6.2.3.5 CBall2RoleDist

函数名	CBall2RoleDist(role)		
实现功能	官方提供的 C++ 函数, 离我方某个球员距离		
输入参数	参数名	参数类型	参数说明
	role	string	角色名
返回参数	dist	number	球离我方某个球员距离 (单位: cm)

6.2.3.6 CBall2PointDir

函数名	CBall2PointDir(our_goal_x,our_goal_y)		
实现功能	官方提供的 C++ 函数, 与我方球门朝向, 球门为球门中心点位置		
输入参数	参数名	参数类型	参数说明
	our_goal_x	string	我方球门中心 X 轴坐标
	our_goal_y	string	我方球门中心 Y 轴坐标
返回参数	dir	number	球与我方球门方向

6.2.3.7 CBall2PointDir

函数名	CBall2PointDir(opp_goal_x,opp_goal_y)		
实现功能	官方提供的 C++ 函数, 与对方球门朝向, 球门为球门中心点位置		
输入参数	参数名	参数类型	参数说明
	opp_goal_x	string	对方球门中心 X 轴坐标
	opp_goal_y	string	对方球门中心 Y 轴坐标
返回参数	dir	number	球与对方球门方向

6.2.4 比赛信息：比赛条件

6.2.4.1 CGameOn

函数名	CGameOn()		
实现功能	官方提供的 C++ 函数，比赛是否开始，获取比赛开始信息		
输入参数	参数名	参数类型	参数说明
	无		
返回参数		Bool	true 开始，false 未开始

6.2.4.2 CNormalStart

函数名	CNormalStart()		
实现功能	官方提供的 C++ 函数，表示裁判盒有没有发开球指令		
输入参数	参数名	参数类型	参数说明
	无		
返回参数		Bool	True 发了，false 没发

6.2.4.3 Cbuf_cnt

函数名	Cbuf_cnt(cond, count)		
实现功能	官方提供的 C++ 函数，当 cond 条件为 true 时控制 true 返回时间		
输入参数	参数名	参数类型	参数说明
	cond	bool	条件判断
	count	number	计数时间
返回参数		bool	条件判断结果

6.3 附件 3: worldmodel lib 库说明

worldmodel_lib 库定义了 WorldModel 类，并通过 WorldModel 类中的方法获得球场上 vision 信息。

6.3.1 get_ball_pos

函数名	get_ball_pos()		
实现功能	获得场上小球的坐标信息		
输入参数	参数名	参数类型	参数说明
	无		
返回参数	Point2f 类型的二维坐标值，返回（9999.0，9999.0）时表示不在场上		

6.3.2 get_our_player_pos

函数名	get_our_player_pos(id)		
实现功能	获得场上己方足球机器人坐标信息		
输入参数	参数名	参数类型	参数说明
	id	int	己方机器人车号
返回参数	Point2f 类型的二维坐标值，返回（9999.0，9999.0）时表示不在场上		

6.3.3 get_opp_player_pos

函数名	get_opp_player_pos(id)		
实现功能	获得场上对方足球机器人坐标信息		
输入参数	参数名	参数类型	参数说明
	id	int	对方机器人车号
返回参数	Point2f 类型的二维坐标值，返回（9999.0，9999.0）时表示不在场上		

6.3.4 get_ball_vel

函数名	get_ball_vel()		
实现功能	获得场上小球速度信息		
输入参数	参数名	参数类型	参数说明
	无		
返回参数	Point2f 类型的二维坐标值（当前帧图像），未获取到时返回(9999.0，9999.0)		

6.3.5 get_our_player_v

函数名	get_our_player_v(id)		
实现功能	获得场上己方足球机器人速度信息		
输入参数	参数名	参数类型	参数说明
	id	int	己方机器人车号
返回参数	Point2f 类型的二维坐标值（当前帧图像），未获取到时返回(9999.0, 9999.0)		

6.3.6 get_opp_player_dir

函数名	get_opp_player_dir(id)		
实现功能	获得场上对方足球机器人方向信息		
输入参数	参数名	参数类型	参数说明
	id	int	对方机器人车号
返回参数	float 类型的角度值，中心为坐标系原点（即球场中心），未获取到时返回 9999.0		

6.3.7 get_our_player_dir

函数名	get_our_player_dir(id)		
实现功能	获得场上己方足球机器人方向信息		
输入参数	参数名	参数类型	参数说明
	id	int	己方机器人车号
返回参数	float 类型的角度值，中心为坐标系原点（即球场中心），未获取到时返回 9999.0		

6.3.8 get_our_goalie

函数名	get_our_goalie()		
实现功能	获得场上己方守门员车号		
输入参数	参数名	参数类型	参数说明
	无		
返回参数	Int 类型的车号值		

6.3.9 get_opp_goalie

函数名	get_opp_goalie()		
-----	-------------------------	--	--

实现功能	获得场上对方守门员车号		
输入参数	参数名	参数类型	参数说明
	无		
返回参数	Int 类型的车号值		

6.3.10 get_our_exist_id

函数名	get_our_exist_id()		
实现功能	获得场上我方球员		
输入参数	参数名	参数类型	参数说明
	无		
返回参数	Bool 类型数组，数组长度为 6，1 号车对应下标 0，2 号车对应下标 1...依此类推		

6.3.11 get_opp_exist_id

函数名	get_opp_exist_id()		
实现功能	获得场上敌方球员		
输入参数	参数名	参数类型	参数说明
	无		
返回参数	Bool 类型数组，数组长度为 6，1 号车对应下标 0，2 号车对应下标 1...依此类推		

6.4 附件 4: utils 工具包说明

PlayerTask 类成员变量说明

6.4.1 orientate

成员名	orientate
变量功能	设定技能小车的任务目标点车头朝向角

变量类型	double
取值范围	角度范围 [-180,180]

6.4.2 target_pos

成员名	Target_pos
变量功能	设定技能小车的任务目标点坐标
变量类型	Point2f
取值范围	X 范围 [-300, 300] Y 范围 [-200, 200]

6.4.3 needKick

成员名	needKick
变量功能	踢球动作执行开关，默认为 false
变量类型	bool
取值范围	true, false

6.4.4 isPass

成员名	isPass
变量功能	是否进行传球，默认为 false
变量类型	bool
取值范围	true, false

6.4.5 needCb

成员名	needCb
变量功能	吸球动作执行开关，默认为 false
变量类型	bool
取值范围	true, false

6.4.6 isChipPass

成员名	isChipPass
变量功能	挑球动作执行开关，默认为 false
变量类型	bool
取值范围	true, false

6.4.7 kickPower

成员名	kickPower
变量功能	踢球力度设置
变量类型	double
取值范围	力度范围[0-127]

6.5 附件 5：开始写自己的代码前你需要知道的信息

6.5.1 足球机器人场地的坐标系建立

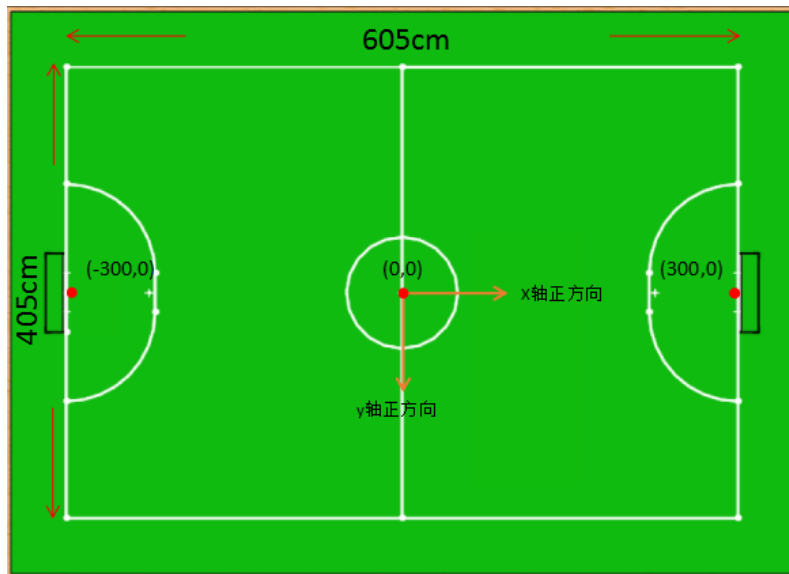


图 6.2 足球场地坐标示意图

如图 6.5.1 所示，场地的中心点为场地坐标系的原点 $(0,0)$ ，以指向敌方球门方向为 x 轴正方向， x 轴正方向顺时针旋转 90 度为 y 轴正方向。这里需要特别注意的地方是：场上己方和敌方半场的选择是随机的，在开始比赛前由敌我双方猜硬币确定； x 轴始终是以指向敌方球门方向为正方向，在策略设计时这一坐标系建立规则都不会改变，无需考虑哪一块半场为己方半场。

球场的标准长度为 600cm ，宽度为 400cm （图 6.2 所示 605cm 中 5cm 为两边线宽余度），球门宽度为 70cm 。根据球场坐标系建立规则，球场上的每一点都可以确定下来，其中敌方球门中点为 $(300,0)$ ，我方球门中点为 $(-300,0)$ 。

6.5.2 足球机器人比赛场景的定义

参考人类足球比赛，我们可以将足球机器人比赛场景分解为中场开球进攻 (KickOff)，中场开球防守 (KickOffDef)，正常比赛 (Normal)，任意球 (IndirectKick)，点球进攻 (PenaltyKick)，点球防守 (PenaltyDef)。其中任意球又可以根据罚球点位置分为前场任意球进攻 (FrontKick)、前场任意球防守 (FrontDef)、中场任意球进攻 (MiddleKick)、中场任意球防守 (MiddleDef)、后场任意球进攻 (BackKick)、后场任意球防守 (BackDef)、角球任意球进攻 (CornerKick)、角球任意球防守

(CornerDef)。

因此整个球场就可以划分为前场、后场、中场、角球区等。这些区域又会根据进攻和防守的区别，有略微的区别。

我们可以调整 lua_scripts\play\Ref 路径下的 OurIndirectKick.lua 和 TheirIndirectKick.lua 文件内的相关参数分别来调整进攻状态和防守状态时各个区域的位置。如图 6.3 和 6.4。



图 6.3

防守

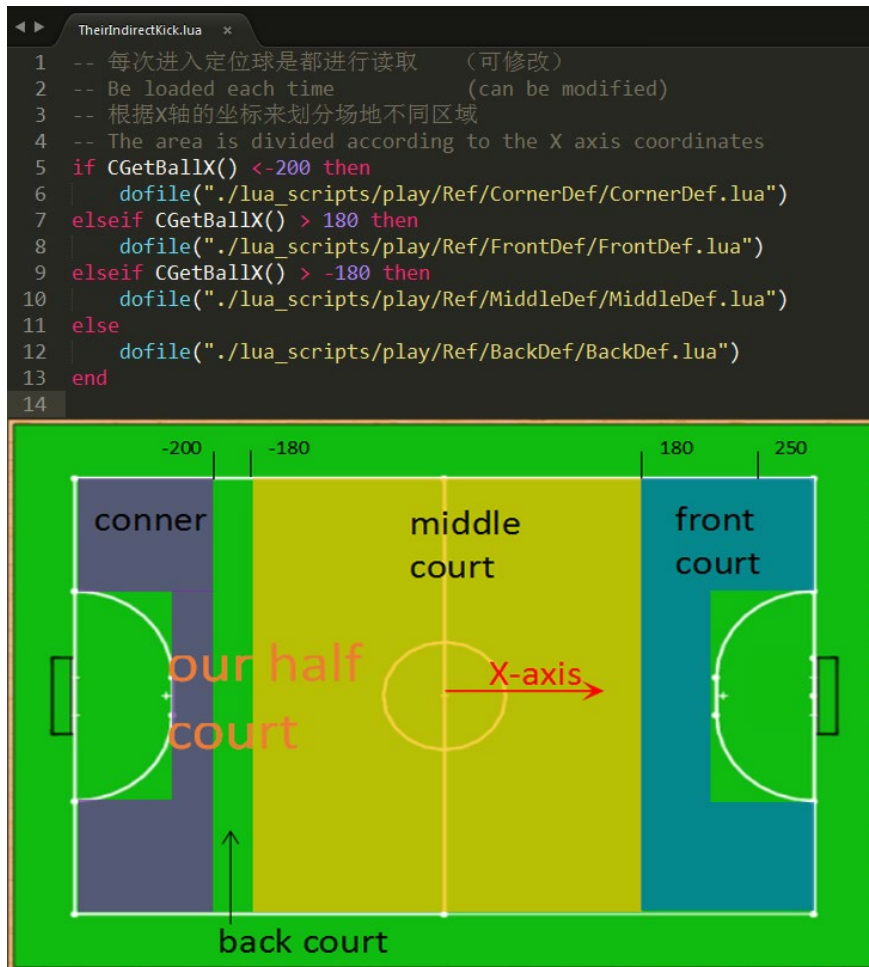


图 6.4

程序中的 CGetBallX 可以获得小球的 X 坐标函数

注意：脚本程序中的球场坐标数值单位为 cm。

6.6 附件 6: 官方 task 函数源码(含源码包)

6.6.1 GetBall 拿球函数

根据场上小球的不同位置配合拿球，源码文件 GetBall.cpp

6.6.2 goalie 守门员函数

只限守门员在禁区内防守，源码文件 goalie.cpp

6.6.3 GoReceivePos 到接球点函数

随机计算接球点，源码文件 GoReceivePos.cpp

6.6.4 GotoPos 跑位函数

指定跑位点，源码文件 GotoPos.cpp

6.6.5 NormalDef 普通防守函数

简单的射门线阻挡防守，源码文件 NormalDef.cpp

6.6.6 PassBall 传球函数

传球 or 射门，源码文件 PassBall.cpp

6.6.7 PenaltyDef 点球防守函数

判断发点球方向，源码文件 PenaltyDef.cpp

6.6.8 PenaltyKick 发点球函数

随机计算发点球方向，源码文件 PenaltyKick.cpp

6.6.9 ReceiveBall 接传球函数

源码文件 ReceiveBall.cpp

6.6.10 RefDef 战术防守函数

前锋和中场战术防守，源码文件 RefDef.cpp

6.6.11 Shoot 射门函数

源码文件 Shoot.cpp

